

## Podemos Decir que....

**Programación funcional** en **Scala** es un paradigma basado en la evaluación de **funciones puras**, como el uso de inmutabilidad y uso de expresiones en lugar de declaraciones.

## Funciones Puras

Una función pura siempre devuelve el mismo resultado con los mismos argumentos y no modifica el estado externo, como lo hemos visto durante todo el semestre.

### Ejemplo:

```
def sumar(x: Int, y: Int): Int = x + y
```

# Inmutabilidad (Evitar var, usar val)

## Inmutabilidad (Evitar var, usar val)

Se evita modificar variables una vez creadas, promoviendo código más seguro y predecible.

**val** → Valor inmutable (no se puede cambiar después de asignarlo).

**var** → Valor mutable (puede cambiarse después de asignarlo).

Ejemplo VAR

```
var edad = 25
```

```
edad = 26 // ☒ Permitido, porque `var` es mutable
```

Ejemplo VAL

```
val nombre = "Scala"
```

```
nombre = "Java" // ✗ Error: No se puede reasignar un valor a `val`
```

# Funciones de Orden Superior

## (Funciones que reciben o retornan funciones)

Como lo hemos visto en otros ejemplo vistos en clases pasadas, Scala permite pasar funciones como parámetros o también permite devolver funciones.

### Ejemplo 1:

```
def operar(f: (Int, Int) => Int, x: Int, y: Int): Int = f(x, y)
val suma = operar((a, b) => a + b, 4, 6) // muestra 10
```

### Ejemplo 2:

```
def crearMultiplicador(factor: Int): Int => Int = {(x: Int) => x * factor}
val doble = crearMultiplicador(2)
val triple = crearMultiplicador(3)
println(doble(5)) // muestra 10
println(triple(5)) // muestra 15
```

# Expresiones Lambda (Funciones anónimas)

Como lo hemos visto en otros ejemplos vistos en clases pasadas el uso del Lambda es fundamental para la programación funcional y scala no es ajeno a ello, por tanto se muestran las 2 formas de usarlo.

## Anónimo

```
object LambdaAnonima { def main(args: Array[String]): Unit = { val numeros = List(10, 20, 30, 40, 50)
```

//lambda anónima se us directamente con el `map`, que ya hemos visto en otros lenguajes

```
val resultado = numeros.map(x => x - 5) // Aplica Función anónima con map,  
println(resultado)} // muestra la lista (5, 15, 25, 35, 45)
```

# Expresiones Lambda (Funciones NO anónimas)

## No anónimo

```
object FuncionNombrada {  
  def restarCinco(x: Int): Int = x - 5 //Gente acá se ve la función explícita que funciona dentro de restar5  
  
  def main(args: Array[String]): Unit = {  
    val numeros = List(10, 20, 30, 40, 50)  
    val resultado = numeros.map(restarCinco) // lo que está en restar5 lo ejecuta con map  
  
    println(resultado) // muestra la Lista(5, 15, 25, 35, 45)  
  }  
}
```

## Diferencias entre los Enfoques

| Enfoque          | Definición                           | Ventajas                |
|------------------|--------------------------------------|-------------------------|
| Lambda Anónima   | Se usa directamente dentro de .map() | Más concisa             |
| Función Nombrada | Se define con def nombreFuncion(...) | Reutilizable, más clara |

# Colecciones de Funciones

Las colecciones funcionales como lo hemos visto en otros lenguajes son fundamentales para la programación funcional.

1. `.map` → Como ya lo vimos antes, `.map` aplica una función a cada elemento de la colección y devuelve una nueva lista con los valores transformados.

```
object MapEjemplo {  
  def main(args: Array[String]): Unit = {val numeros = List(1, 2, 3, 4, 5)  
  
    val cuadrados = numeros.map(x => x * x)    // con esto le decimos que aplique una transformación (elevar al cuadrado)  
    println(cuadrados)          // muestra la Lista(1, 4, 9, 16, 25)  
  }  
}
```

# Colecciones de Funciones

Las colecciones funcionales como lo hemos visto en otros lenguajes son fundamentales para la programación funcional.

2. Filter → Como ya lo vimos antes, filter() selecciona solo aquellos elementos que cumplen una condición.

```
object FilterEjemplo { def main(args: Array[String]): Unit = { val numeros = List(1, 2, 3, 4, 5, 6)

    val pares = numeros.filter(x => x % 2 == 0) // Filtramos los números pares
    println(pares)    // muestra la Lista(2, 4, 6)
  }
}
```



# Colecciones de Funciones

Las colecciones funcionales como lo hemos visto en otros lenguajes son fundamentales para la programación funcional.

2. reduce → Como ya lo vimos antes, reduce() reduce una colección a un único valor, es decir, aplica una operación acumulativa sobre los elementos.

```
object ReduceEjemplo { def main(args: Array[String]): Unit = { val numeros = List(1, 2, 3, 4, 5)
```

```
    val sumaTotal = numeros.reduce((x, y) => x + y) // Sumamos todos los valores
```

```
    println(sumaTotal) // Adivinen cuanto da: 15
```

```
  }  
}
```

# Colecciones de Funciones

Las colecciones funcionales como lo hemos visto en otros lenguajes son fundamentales para la programación funcional.

2. `fold()` → Como ya lo vimos antes, `fold()` es similar a `reduce()`, pero indicándole un valor inicial, es decir, permite un **valor inicial**, antes de aplicar la operación acumulativa.

```
object FoldEjemplo {  
  def main(args: Array[String]): Unit = { val numeros = List(1, 2, 3, 4, 5)
```

```
    val sumaConValorInicial = numeros.fold(10)((x, y) => x + y) // gente fold iniciado en 10 para la suma de todos los valores
```

```
    println(sumaConValorInicial) // muestra :25  
  }  
}
```

# Colecciones de Funciones

Las colecciones funcionales como lo hemos visto en otros lenguajes son fundamentales para la programación funcional.

2. flatmap() → Como ya lo vimos antes, el flatmap, aplica una función a cada elemento, pero en lugar de devolver listas dentro de listas, **las aplanar en una sola lista**.

```
object FlatMapEjemplo {  
  def main(args: Array[String]): Unit = { val palabras = List("el", "JuanGui")
```

```
    val caracteres = palabras.flatMap(palabra => palabra.toList) // Se Descompone c/palabra en caracteres
```

```
    println(caracteres) // muestra la Lista(e, l, J, u, a, n, G, u, i)  
  }  
}
```

| Método    | Propósito                     | Ejemplo                          |
|-----------|-------------------------------|----------------------------------|
| map()     | Transformar cada elemento     | Elevar al cuadrado               |
| filter()  | Seleccionar ciertos elementos | Filtrar números pares            |
| reduce()  | Reducir a un único valor      | Suma total                       |
| fold()    | Reduce con valor inicial      | Suma con valor inicial           |
| flatMap() | Transformar y aplanar listas  | Convertir palabras en caracteres |

# Práctica de Scala

Realizar un ejercicio en Scala que realice procesamiento funcional de datos de ventas de carros en el valle de aburrá.  
Aplicar programación funcional para filtrar, transformar y analizar información sobre ventas de carros.

**Un concesionario almacena las ventas de automóviles en una lista de case class, con la siguiente estructura (Ojo solo es sugerida):**

```
case class Vehiculo(id: Int, marca: String, modelo: String, año: Int, precio: Double)
```

```
case class Venta(id: Int, vehiculo: Vehiculo, vendedor: String, cliente: String)
```

**Lista de ventas sugeridas( Quemada):**

```
val ventas = List( Venta(1, Vehiculo(101, "Toyota", "Corolla Cross", 2025, 7000), "Juan", "Gui"),
```

```
Venta(2, Vehiculo(102, "Honda", "Civic", 2023, 25000), "Luis", "P"),
```

```
Venta(3, Vehiculo(103, "Ford", "Mustang", 2021, 45000), "Sara", "Falta"),
```

```
Venta(4, Vehiculo(104, "Tesla", "Model 3", 2022, 60000), "Sofí", "a"),
```

```
Venta(5, Vehiculo(105, "Chevrolet", "Camaro", 2020, 38000), "Diego", "Roberto"))
```

# ¿Qué hay que entregar?

- ✓ Calcular el total de ventas por marca usando groupBy y map.
- ✓ Filtrar vehículos vendidos con precio mayor a 30 M (COP) usando filter.
- ✓ Encontrar el vehículo más caro vendido usando reduce en el último año vigente.
- ✓ Ordenar las ventas por año del vehículo de más reciente a más antiguo usando sortBy.
- ✓ Generar un reporte en formato JSON con las estadísticas generadas.

## **Restricciones de código:**

- No usar **bucles tradicionales (for, while)**, solo métodos funcionales.

El código debe ser organizado con funciones reutilizables y sin comentarios para su sustentación